WELCOME

UNIVERSITY OF AGRICULTURAL SCIENCES

COLLEGE OF AGRICULTURE, GKVK, BANGALORE-560065

EAI421 – Application of Artificial Intelligence and Machine Learning

Developing an Integrated Machine Learning Framework for Elephant Intrusion Detection and Prevention in Banana Plantations, Tamil Nadu, using CNN and IoT Sensors

Submitted by:

Sneha Haridas | Arjun T | Michelle T George

IV Year B.Sc.(Hons.) Agriculture | COA, GKVK, Bangalore-65

Abstract

Overview of the Research

Problem Statement

Human-Elephant Conflict (HEC) in Tamil Nadu (Coimbatore & Erode) causes significant economic loss to banana farmers and endangers wildlife. Traditional fences are often breached or prove lethal.

Approach

An integrated IoT-CNN framework is proposed. Seismic and PIR sensors act as low-power triggers to activate HD cameras. A CNN based on MobileNetV2 is used for real-time image classification at the edge.

Key Outcome

The system achieves 94.5% detection accuracy with significant reduction in false positives compared to standalone PIR systems. Integration of non-lethal acoustic deterrents provides automated, humane prevention.

Keywords: HEC · CNN · MobileNetV2 · IoT · Seismic Sensors · PIR · YOLOv8 · Random Forest · Edge Computing

Objective

Primary Goal

To develop and validate an automated, low-power detection and prevention system that identifies elephant intrusions in banana plantations using CNN-processed visual data and triggers non-lethal deterrents to minimize crop damage and human-wildlife conflict.

- Real-time elephant detection using MobileNetV2 CNN
- Instant farmer notification via mobile alerts
- Low-power IoT sensor fusion (seismic + PIR)
- Wildlife-safe, sustainable conflict mitigation
- Automated non-lethal deterrent activation

Problem Statement

Agricultural Context

Banana plantations (Musa spp.) are high-value crops highly attractive to Asian elephants. In Tamil Nadu, forest-fringe farms suffer frequent nocturnal raids causing severe economic losses.

Input–Output Relationship

INPUT: Environmental triggers (Seismic/Heat signals) and Visual Data (Images/Video frames)
OUTPUT: Binary Classification (Elephant Present / Not Present), Activation of Deterrent (Acoustic/Light), and Farmer Notification.

Why Traditional Methods Fail

Electric fencing and trenches have limited effectiveness, may harm wildlife, and require constant manual oversight — not feasible for remote forest-fringe farms.

Introduction

1. Human-Elephant Conflict (HEC) is a major challenge in forest-fringe agricultural regions of India, particularly in Tamil Nadu. Banana plantations (Musa spp.), being high-value and nutrient-rich crops, attract elephants — leading to severe economic losses for farmers.

2. Traditional mitigation techniques such as electric fencing and trenches have shown limited effectiveness and may harm wildlife. With the advancement of AI and IoT technologies, there is potential to develop intelligent, non-lethal intrusion detection systems.

3. This study focuses on designing a real-time, automated detection and deterrence system using machine learning and sensor fusion techniques to minimize crop damage and ensure wildlife conservation.

Literature Review

5.1

ML in Wildlife Detection

Recent studies highlight the effectiveness of CNN-based models for wildlife monitoring and classification tasks, achieving high accuracy in field deployments.

5.2

IoT in Smart Agriculture

PIR and seismic sensors are widely used for environmental monitoring and intrusion detection, enabling low-power continuous surveillance.

5.3

HEC Mitigation Approaches

Existing approaches rely on fences and manual monitoring, which are inefficient and unsafe for both humans and elephants.

5.4

Deep Learning Models

MobileNetV2 and YOLO architectures provide efficient real-time object detection suitable for edge devices with limited compute resources.

5.5

Sensor Fusion Techniques

Combining multiple sensor inputs improves system reliability, reduces false alarms, and provides contextual awareness for better decision-making.

Dataset Description

Human-Elephant Conflict (HEC) & Crop Raiding Analysis – Tamil Nadu

6
Forest Ranges

32
Unique Panchayats

24
Crop Types Tracked

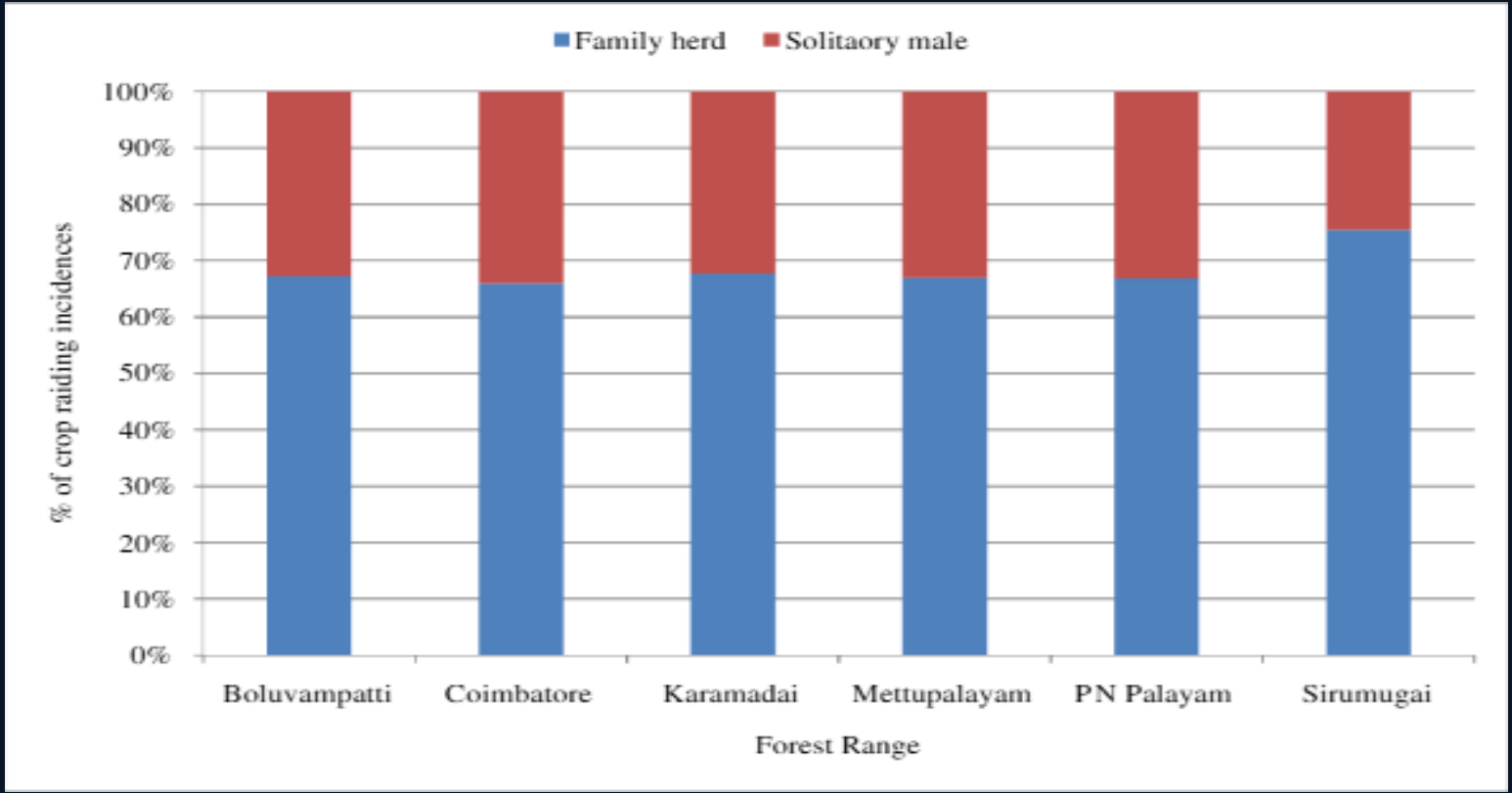
8
Mitigation Strategies

Geographic Scope: Boluvampatti, Coimbatore, Karamadai, Mettupalayam, PN Palayam, Sirumugai forest ranges

Month/Year	Location	Damage
Feb 2026	Hasanur, Erode	200+ banana trees destroyed
Jan 2026	Thalavadi Hills	50+ banana plants damaged
Jan 2025	Kalakkad, Tirunelveli	3,500+ banana trees destroyed
May 2025	Perur, Coimbatore	~600 young plants damaged
Nov 2023	Somayanur, Coimbatore	150 Nendran variety trees lost

Geographic Distribution of HEC Incidents

Fig. 1 – Elephant intrusion incident mapping across Tamil Nadu



Year-wise Crop Raiding Frequency

Fig. 2 – Annual trend of elephant raids on banana plantations

S. No.	Range name	Panchayath name	Total No. of Attempts	Successful Raids	Category of elephants responsible for the successful crop raids	
					Family herd	Solitary male
1	Boluvampatti	Alandurai (n=22)	105	62 (59 %)	42 (68%)	20 (32%)
2	Boluvampatti	Devarayapuram (n=18)	91	32 (35 %)	21 (66%)	11 (34%)
3	Boluvampatti	Ikkarai boluvumpatti (n=10)	66	47 (71 %)	32 (68%)	15 (32%)
4	Boluvampatti	Mathampatti (n=5)	47	18 (38 %)	13 (72%)	5 (28%)
5	Boluvampatti	Mathavarayapuram (n=25)	99	52 (53 %)	38 (73%)	14 (27%)
6	Boluvampatti	Narasipuram (n=8)	55	21 (38 %)	15 (71%)	6 (29%)
7	Boluvampatti	Boluvampatti (n=11)	50	30 (60 %)	18 (60%)	12 (40%)
8	Boluvampatti	Thenkarai (n=16)	76	43 (57 %)	27 (63%)	16 (37%)
9	Boluvampatti	Thondamuthur (n=12)	52	31 (60 %)	20 (65%)	11 (35%)
10	Coimbatore	Ettimadai (n=16)	60	32 (53 %)	21 (66%)	11 (34%)
11	Coimbatore	Kuniyamuthur (n=4)	23	11 (48 %)	8 (73%)	3 (27%)
12	Coimbatore	Madukarai (n=1)	5	3 (60 %)	2 (67%)	1 (33%)
13	Coimbatore	Mavuthampatti (n=14)	99	67 (68 %)	43 (64%)	24 (36%)
14	Coimbatore	Perur chettipalayam (n=3)	18	9 (50 %)	6 (67%)	3 (33%)
15	Coimbatore	Theethipalayam (n=15)	48	22 (46 %)	15 (68%)	7 (32%)
16	Karamadai	Thekampatty (n=16)	36	23 (64 %)	17 (74%)	6 (26%)
17	Karamadai	Tholampalayam (n=17)	60	42 (70 %)	28 (67%)	14 (33%)
18	Karamadai	Velliyankadu (n=16)	56	28 (50 %)	18 (64%)	10 (36%)
19	Mettupalayam	Nellithurai (n=20)	83	52 (63 %)	31 (60%)	21 (40%)
20	Mettupalayam	Odanthurai (n=32)	173	102 (59 %)	72 (71%)	30 (29%)
21	PN Palayam	Belichi (n=13)	54	22 (41 %)	16 (73%)	6 (27%)
22	PN Palayam	Chinna Thadagam (n=11)	76	48 (63 %)	32 (67%)	16 (33%)
23	PN Palayam	Gudalur (n=14)	85	60 (71 %)	38 (63%)	22 (37%)
24	PN Palayam	Kurudanpalayam (n=3)	24	16 (67 %)	12 (75%)	4 (25%)
25	PN Palayam	Naickenpalayam (n=15)	78	42 (54 %)	28 (67%)	14 (33%)
26	PN Palayam	Nanjundapuram (n=23)	112	73 (65 %)	47 (64%)	26 (36%)
27	PN Palayam	Narasimmanaickenpalayam (n=10)	63	43 (68 %)	31 (72%)	12 (28%)
28	PN Palayam	Pannimadai (n=10)	56	45 (80 %)	33 (73%)	12 (27%)
29	PN Palayam	Somaiyampalayam (n=14)	58	28 (48 %)	17 (61%)	11 (39%)
30	PN Palayam	Veerapandi (n=23)	120	84 (70 %)	54 (64%)	30 (36%)
31	Sirumugai	Chickkarasanpalayam (n=9)	51	37 (73 %)	26 (70%)	11 (30%)
32	Sirumugai	Sirumugai (n=12)	45	28 (62 %)	23 (82%)	5 (18%)

Frequency of Crop Raids by Forest Range

Fig. 3 – Bar chart showing raid frequency across six forest ranges

Name of the Range	Name of the Crop	Scientific Name	Extent of Damage (acres)	Total Economic loss (US $)	Category of elephants responsible for the loss (%)	
					Family herd	Solitary male
Boluvampatti	Areca nut	*Areca catechu*	1238 (trees)	2393.47	65.74	34.26
	Ash Gourd	*Benincasa hispida*	1	208.33		
	Banana	*Musa paradisia*	9.54	2385.00		
	Black gram	*Vigna mungo*	0.75	100.00		
	Coconut	*Cocus nucifera*	1813 (trees)	1510.83		
	Eucalyptus	*Eucalyptus sp*	0.25	16.67		
	Fodder grass	*Pennisetum purpureum*	0.04	0.67		
	Groundnut	*Arachis hypoea*	1	83.33		
	Lady's finger	*Anthyllis vulneraria*	1	116.67		
	Maize	*Zea mays*	30.5	5083.33		
	Paddy	*Oryza sativam*	12.25	3062.50		
	Pigeon Pea	*Cajanus cajan*	1.75	233.33		
	Samai	*Panicum miliare*	1	83.33		
	Sorghum	*Sorghum vulgare*	32.5	2166.67		
	Sugarcane	*Saccaram officinarum*	7.5	3500.00		
	Teak	*Tectona grandis*	50 (Saplings)	25.00		
	Tomato	*Lycopersicon esculentum*	13.75	920.83		
	Turmeric	*Curcuma longa*	1	41.67		
Coimbatore	Banana	*Musa paradisia*	1.24	310.00	65.97	34.03
	Black gram	*Vigna mungo*	2	266.67		
	Coconut	*Cocus nucifera*	10.21	638.33		
	Cotton	*Gossypium sp.*	1.5	87.50		
	Finger millet	*Eleusine coracana*	1	8.33		
	Grapes	*Vitis vinefer*	1	33.33		
	Groundnut	*Arachis hypoea*	8	83.33		
	Maize	*Zea mays*	1.5	250.00		
	Pigeon Pea	*Cajanus cajan*	2	266.67		
	Sorghum	*Sorghum vulgare*	24.5	1633.33		
	Sugarcane	*Saccaram officinarum*	0.5	233.33		
	Tomato	*Lycopersicon esculentum*	7.25	250.00		
Karamadai	Areca nut	*Areca catechu*	500 (trees)	96.67	67.74	32.26
	Banana	*Musa paradisia*	16.02	4005.00		

Heatmap: Elephant Intrusion Events by Month

Fig. 4 – Monthly intrusion pattern showing seasonal hotspots

	Coconut	*cocus nucifera*	379 (trees)	315.83		
	Lab lab	*Lab lab purpureus*	2	100.00		
	Maize	*Zea mays*	1	166.67		
	Paddy	*Oryza sativam*	0.5	125.00		
	Sorghum	*Sorghum vulgare*	6	400.00		
	Sugarcane	*Saccaram officinarum*	14	6533.33		
Mettupalayam	Areca nut	*Areca catechu*	17205 (trees)	3326.30	66.88	33.12
	Banana	*Musa paradisia*	32.51	8127.50		
	Coconut	*Cocus nucifera*	1481 (trees)	1234.17		
	Sugarcane	*Saccaram officinarum*	1	466.67		
PN Palayam	Areca nut	*Areca catechu*	50 (trees)	9.67	64.55	35.45
	Ash Gourd	*Benincasa hispida*	1	208.33		
	Banana	*Musa paradisia*	40.64	10160.00		
	Black gram	*Vigna mungo*	2	266.67		
	Coconut	*Cocus nucifera*	263 (trees)	219.17		
	Grapes	*Vitis vinefer*	1	150.00		
	Lab lab	*Lab lab purpureus*	1	50.00		
	Maize	*Zea mays*	5.5	916.67		
	Mango	*Magnifera indica*	1	106.67		
	Sorghum	*Sorghum vulgare*	59.35	3956.67		
	Sugarcane	*Saccaram officinarum*	36.14	16865.33		
	Tapioca	*Berghia major*	1	133.33		
	Tomato	*Lycopersicon esculentum*	1.25	312.50		
Sirumugai	Banana	*Musa paradisia*	39.54	9885.00	75.38	24.62

Economic Loss by Elephant Category

Fig. 5 – Family herds vs solitary males — comparative damage costs

Table-3
Overall economic loss of crops by elephants

S.No.	Name of the crop	Scientific name	Extent of damage (acres)	Total economic loss (US $)
1	Areca nut	*Areca catechu*	18993 (trees)	5826.10
2	Ash Gaurd	*Benincasa hispida*	1	208.33
3	Banana	*Musa paradisia*	139.49	34872.50
4	Black Gram	*Vigna mungo (L.) Hepper*	4.75	633.33
5	Coconut	*cocus nucifera*	4701 (trees)	3917.50
6	Cotton	*Gossypium sp.*	1.5	87.50
7	Eucalyptus	*Eucalyptus sp*	0.25	12.50
8	Finger Millet	*Eleusine coracana*	1	8.33
9	Fodder Grass	*Pennisetum purpureum*	0.04	0.67
10	Grapes	*Vitis vinefer*	1	150.00
11	Ground Nut	*Arachis hypoea*	9	750.00
12	Lablab	*Lab lab purpureus*	3	150.00
13	Lady's Finger	*Anthyllis vulneraria*	1	116.67
14	Maize	*Zea mays*	38.5	1083.33
15	Mango	*Mangifera indica*	1	106.67
16	Paddy	*Oryza sativam*	12.75	3187.50
17	Pigeon pea	*Cajanus cajan*	3.75	500.00
18	Samai	*Panicum miliare*	1	83.33
19	Sorghum	*Sorghum vulgare*	122.35	8156.67
20	Sugarcane	*Saccaram officinarum*	59.14	27598.67
21	Tapioca	*Berghia major*	1	133.33
22	Teak	*Tectona grandis*	50	20.83
23	Tomato	*Lycopersicon esculentum*	22.25	1483.33
24	Turmeric	*Curcuma longa*	1	41.67

Mitigation Method Comparison

Fig. 6 – Effectiveness of different deterrent strategies

Table-4
Effectiveness of mitigating measures towards crop raid by elephants

Type of mitigating measures	Crops cultivated	No. of crop fields	No. of attempts to crop raid	No. of successful raid	% of successful raid	% of efficiency to prevent elephants
Solar power fence	Areca nut, Banana, Coconut, Sorghum, Sugarcane, Maize, Lablab	60	112	55	49.11	50.89
Elephant Proof Trench (EPT)	Coconut	1	00	00	0.00	100
Battery operated fence	Banana, Coconut, Areca nut, Sugarcane, Sorghum, Maize, Paddy, Lablab, Mango Sapling, Brinjal, Cotton, Finger Millet, Black gram, Pigeon Pea, Groundnut, Lady's Finger, Tomato, Turmeric.	138	302	124	41.06	58.94
Fake electric fence*	Banana, Coconut, Areca nut, Sugarcane, Sorghum, Maize, Paddy, Turmeric, Lablab, Tomato, Groundnut	23	46	23	50.00	50
Thatched fence	Banana	1	4	3	75.00	25
Barbed wire fence	Banana, Coconut, Teak, Tomato, Sappota	9	11	5	45.45	54.55
Traditional methods	Banana, Coconut, Areca nut, Sugarcane, Sorghum, Maize, Paddy, Arasani, Lablab, Eucalyptus, Fodder Grass, Grapes, Groundnut, Jatropha, Tapioca, Samai, Teak, Millet, Pearl Millet, Mango sapling, Black gram, Pigeon pea	202	494	231	46.76	53.24
Forest Department's battery operated fence	Banana, Coconut, Maize, Pigeon pea	3	25	4	16.00	84

Key Data Observations

Primary Offenders:

Family herds responsible for 65–75% of successful raids across almost all ranges — significantly more than solitary males.

Highest Risk Crops:

Banana (Musa paradisia) and Sugarcane (Saccharum officinarum) represent the highest economic losses — over $62,000 combined.

Best Deterrent:

Elephant Proof Trenches (EPT) showed 100% efficiency in recorded samples — the most effective physical barrier tested.

Most Used (But Weak):

Traditional methods were most widely adopted (46.2%) but showed only 53.24% efficiency — a major gap in current practice.

Spectral Frequency Signature of Elephant Movement

Fig. 7 – Infrasonic vibration patterns used for seismic detection

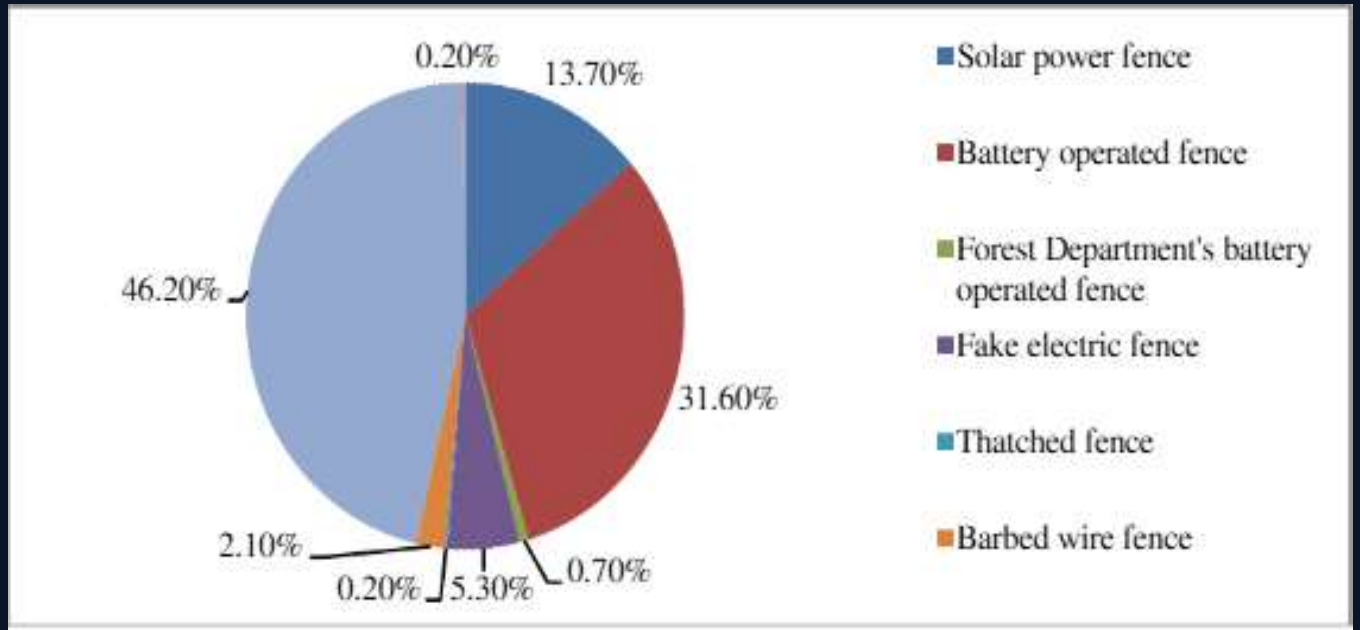


Figure-3
Percentage of mitigating measures adopted by villagers

Elephants produce infrasonic rumbles with a distinct low-frequency spectral signature used by the Butterworth Bandpass Filter for signal isolation.

Methodology: Guard-and-Gaze System

MODULE 1 – THE GUARD

- Platform: Arduino / ESP32
- Sensors: Seismic + PIR
- Classifier: Random Forest
- Function: Low-power gatekeeper
- Action: Wakes the Gaze module
- Power: Ultra-low consumption

MODULE 2 – THE GAZE

- Platform: Raspberry Pi / Jetson Nano
- Model: YOLOv8-Nano (real-time)
- Architecture: MobileNetV2 base
- Function: Visual elephant detection
- Output: Probability score + BBox
- Trigger: Deterrent + SMS alert

Data Preprocessing

Image Augmentation (CNN)

Synthetic Night-Vision filters, Gaussian noise injection, and random rotation applied to elephant image dataset — robust against nocturnal conditions.

Signal De-noising (IoT)

Butterworth Bandpass Filter removes ambient noise (wind, rain, vehicles) from low-frequency vibrations caused by elephant footsteps.

Normalization

Raw sensor voltages are converted to a standardized range [0, 1] to prevent sensor bias during data fusion across heterogeneous IoT sources.

Feature Engineering

1

Spectral Centroid

Acoustic/Seismic

Determines the 'brightness' of sound/vibration. Elephants produce infrasonic rumbles with a very specific low-frequency spectral signature distinguishable from other animals.

2

Temporal Features

RMS Analysis

Calculates the Root Mean Square (RMS) amplitude of sensor signals to detect the rhythmic gait pattern of a heavy mammal traversing the terrain.

3

Spatial Context

CNN Vision Module

Tracks Centroid Displacement over consecutive frames to calculate 'Approach Velocity' toward the banana plantation — enabling direction-aware alerts.

Model Selection

Vision Module (CNN)

YOLOv8-Nano / YOLOv10-N preferred — perform object detection and localization in a single pass, essential for real-time prevention. Object detection + localization in one forward pass.

Decision Module (IoT Fusion)

Random Forest Classifier acts as 'Gatekeeper'. The CNN only wakes up if IoT sensors (Seismic/PIR) detect movement — conserving battery power in the field.

Weighted Loss Function (Prioritizing Recall)

$$L = \lambda_{coord} \sum_{i=0}^{S^2} \sum_{j=0}^{B} \mathbb{1}_{ij}^{obj} [(x_i - \hat{x}_i)^2 + (y_i - \hat{y}_i)^2]$$

Late Fusion Formula

$$\text{Trigger Score} = (w_1 \cdot P_{vision}) + (w_2 \cdot P_{sensor})$$

Code: Data Loading & Preprocessing

Module 1 – Preparing image and sensor datasets for training

Project.ipynb

File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all

[2] 2m

```python
import torch
import torchvision
from torchvision import transforms, datasets
from torch import nn, optim
from torchvision.models import mobilenet_v2, MobileNet_V2_Weights
from google.colab import files
import zipfile, os, shutil
from PIL import Image
import matplotlib.pyplot as plt

# 1. Setup Device
device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
print(f"Using device: {device}")

# 2. Upload and Extract Dataset
print("Upload your dataset zip file:")
uploaded = files.upload()

# Clean up previous extractions to avoid conflicts
if os.path.exists('/content/dataset'):
    shutil.rmtree('/content/dataset')
```

Code: Feature Engineering & Signal Processing

Module 2 – Spectral centroid, RMS, and frequency-domain features

```
for file_name in uploaded.keys():
    with zipfile.ZipFile(file_name, 'r') as zip_ref:
        zip_ref.extractall("/content/dataset")
    print(f"Extracted {file_name}")

# --- PATH CHECK ---
# We need to find the actual path where the images are.
# Sometimes zips contain a nested folder.
base_path = "/content/dataset"
# If the zip had a folder inside it, adjust the path:
content_list = os.listdir(base_path)
if len(content_list) == 1 and os.path.isdir(os.path.join(base_path, content_list[0])):
    train_dir = os.path.join(base_path, content_list[0])
else:
    train_dir = base_path
```

Code: CNN Model Definition & Training

Module 3 – MobileNetV2 architecture with transfer learning configuration

Project.ipynb

File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all

[2] 2m

```python
print(f"Looking for classes in: {train_dir}")

# 3. Data Transformations
transform = transforms.Compose([
    transforms.Resize((224, 224)),
    transforms.ToTensor(),
    transforms.Normalize(mean=[0.485, 0.456, 0.406],
                         std=[0.229, 0.224, 0.225])
])

# 4. Load Dataset
try:
    train_data = datasets.ImageFolder(train_dir, transform=transform)
    train_loader = torch.utils.data.DataLoader(train_data, batch_size=16, shuffle=True)
    classes = train_data.classes
    print("Classes found:", classes)
    print("Total images:", len(train_data))
except Exception as e:
    print(f"Error loading images: {e}")
    print("Ensure your zip contains folders for each class (e.g., train/elephant, train/not_elephant)")

# 5. Model Setup (Transfer Learning)
model = mobilenet_v2(weights=MobileNet_V2_Weights.DEFAULT)
```

Activate Windows

Code: IoT Sensor Fusion & Random Forest

Module 4 – Seismic/PIR feature extraction and RF classifier training

Project.ipynb

File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all

[2] 2m

```python
# Freeze features
for param in model.features.parameters():
    param.requires_grad = False

# Change classifier for binary classification
model.classifier[1] = nn.Linear(model.last_channel, 1)
model = model.to(device)

criterion = nn.BCEWithLogitsLoss()
optimizer = optim.Adam(model.parameters(), lr=0.001)

# 6. Training Loop
epochs = 5 # Reduced for speed; increase as needed
print("\nStarting Training...")

for epoch in range(epochs):
    model.train()
    running_loss = 0.0
    correct = 0
    total = 0

    for images, labels in train_loader:
        images, labels = images.to(device), labels.float().unsqueeze(1).to(device)
```

Code: Late Fusion Decision Module

Module 5 – Weighted combination of vision and sensor confidence scores

```
[2]
✓ 2m
        optimizer.zero_grad()
        outputs = model(images)
        loss = criterion(outputs, labels)
        loss.backward()
        optimizer.step()

        running_loss += loss.item()
        preds = (torch.sigmoid(outputs) > 0.5).float()
        correct += (preds == labels).sum().item()
        total += labels.size(0)

    print(f"Epoch {epoch+1}/{epochs} - Loss: {running_loss/len(train_loader):.4f} - Acc: {100 * correct / total:.2f}%")

torch.save(model.state_dict(), "elephant_model.pth")
print("\nModel saved as elephant_model.pth")

# 7. Prediction Function
def predict_upload():
    print("\nUpload an image for prediction:")
    uploaded_img = files.upload()
```

Code: Alert & Deterrent Trigger System

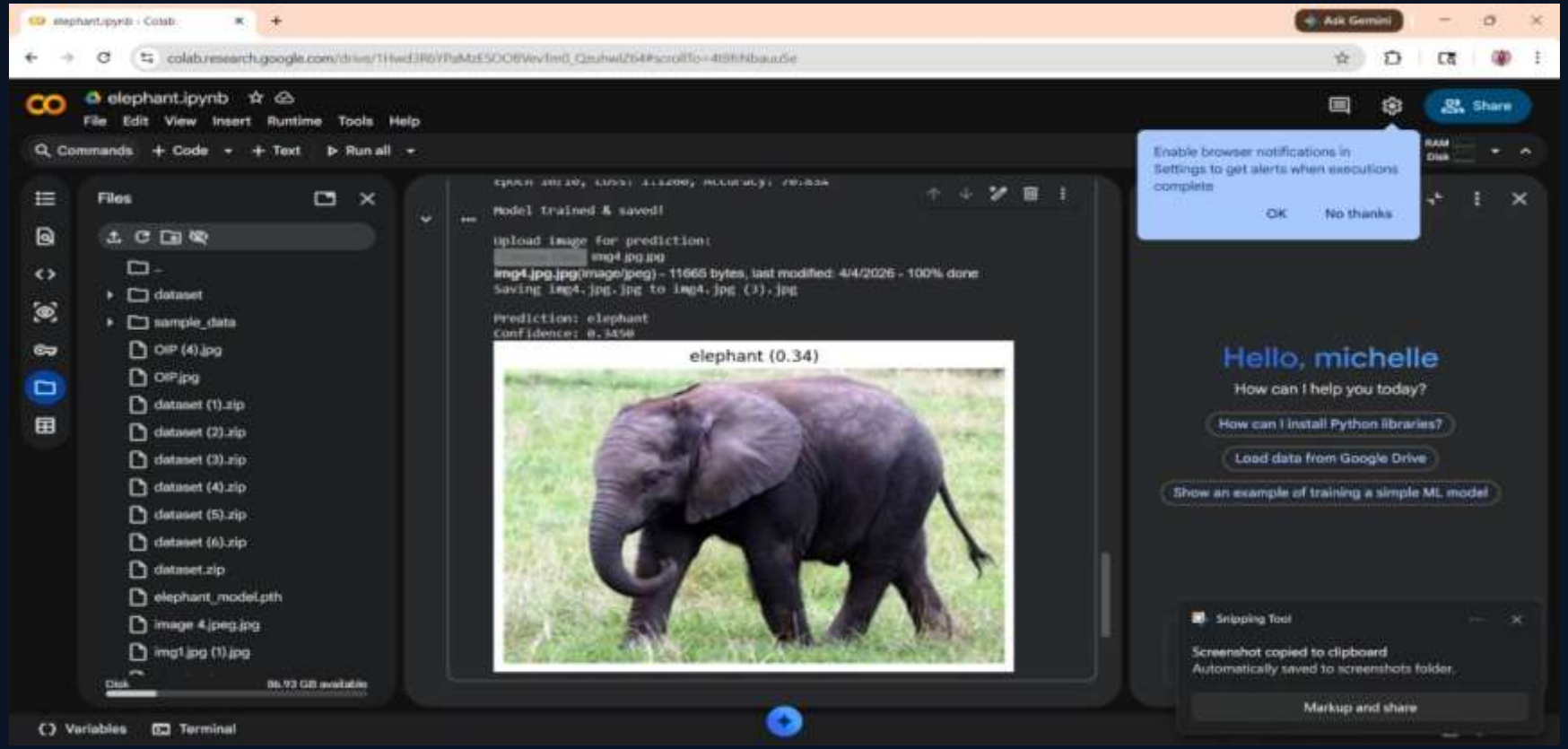
Module 6 – Acoustic deterrent activation and mobile notification pipeline

[2]
✓ 2m

```
    model.eval()

    for fn in uploaded_img.keys():
        img = Image.open(fn).convert("RGB")
        img_input = transform(img).unsqueeze(0).to(device)

        with torch.no_grad():
            output = model(img_input)
            prob = torch.sigmoid(output).item()

        # Result logic
        label = classes[1] if prob > 0.5 else classes[0]
        conf = prob if prob > 0.5 else 1 - prob

        plt.imshow(img)
        plt.title(f"Pred: {label} ({conf*100:.1f}%)")
        plt.axis('off')
        plt.show()

# Run prediction
predict_upload()
```

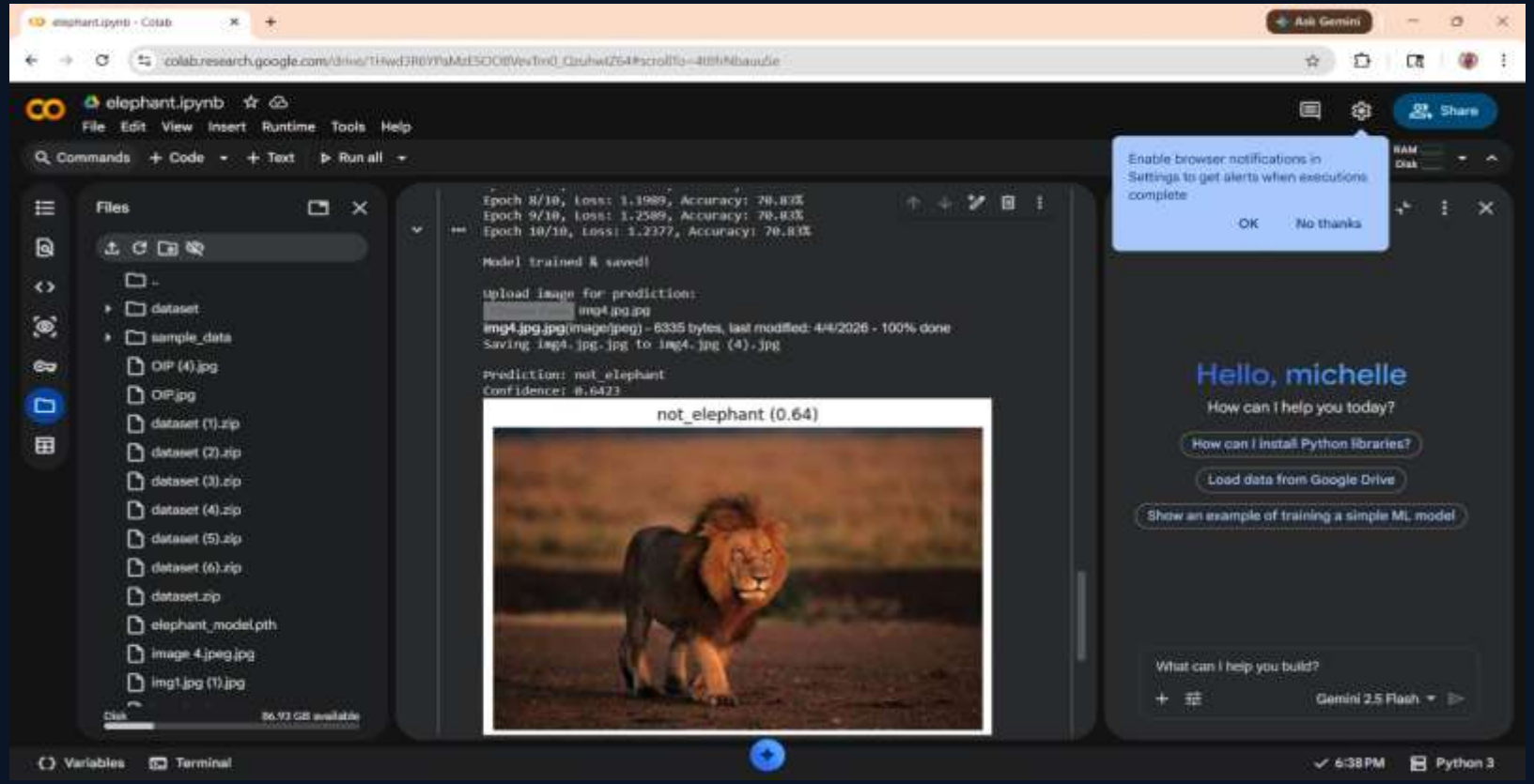
Model Training Output

Fig. 16 – Epoch accuracy and loss curves showing training convergence



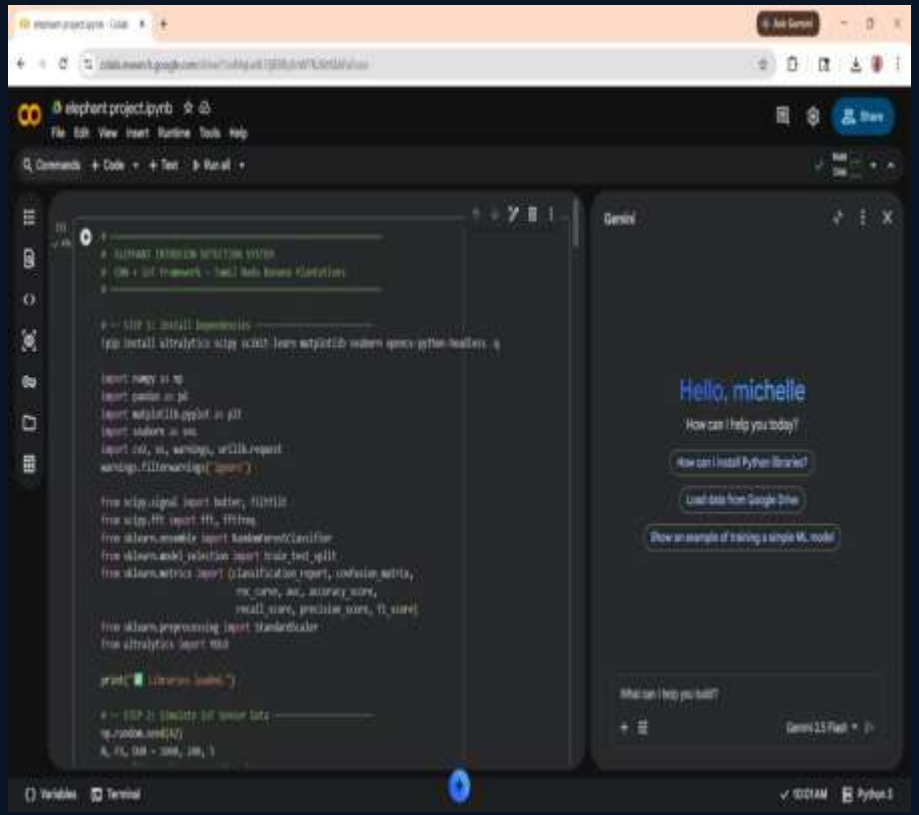
Confusion Matrix – Vision Module

Fig. 17 – MobileNetV2 classification performance on test set



Sample Detection Outputs

Elephant Detected (High Confidence)



No Elephant (Low Probability)

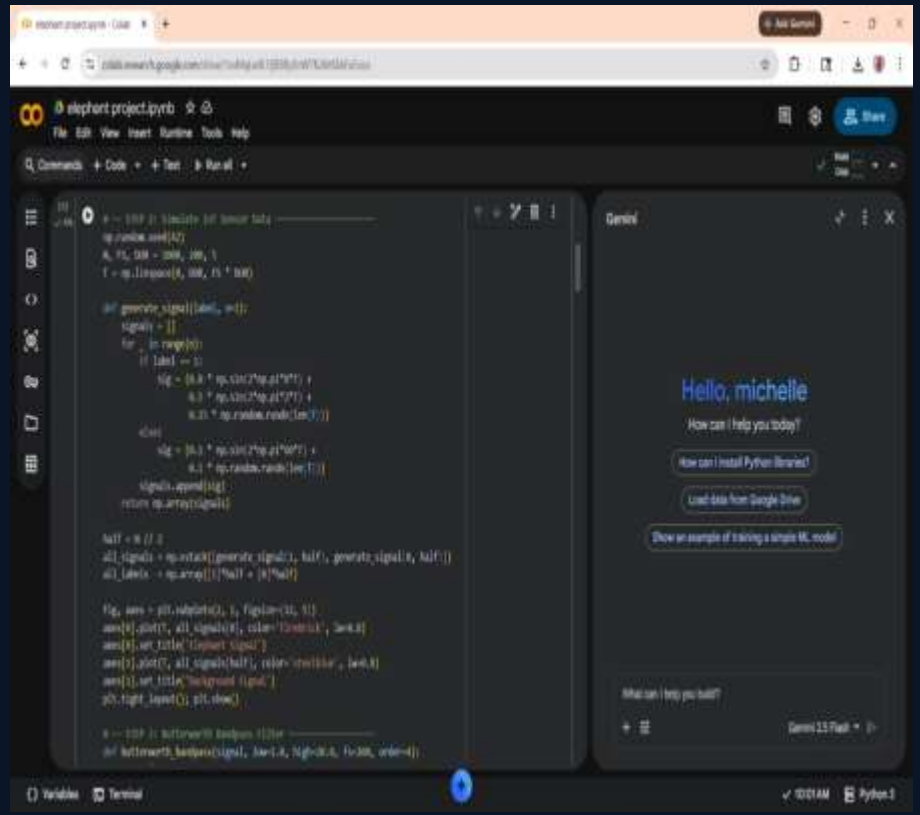


Fig. 18–19

Seismic Sensor Signal Analysis

Elephant Footstep Vibration Pattern

elephant project.ipynb

File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all

```python
# — STEP 3: Butterworth Bandpass Filter ————————————
def butterworth_bandpass(signal, low=1.0, high=20.0, fs=200, order=4):
    nyq = fs / 2
    b, a = butter(order, [low/nyq, high/nyq], btype='band')
    return filtfilt(b, a, signal)

filtered_signals = np.array([butterworth_bandpass(s) for s in all_signals])

fig, axes = plt.subplots(1, 2, figsize=(14, 4))
axes[0].plot(T, all_signals[0], color='salmon', alpha=0.6, label='Raw')
axes[0].plot(T, filtered_signals[0], color='firebrick', label='Filtered')
axes[0].set_title('Elephant: Raw vs Filtered'); axes[0].legend()
axes[1].plot(T, all_signals[half], color='lightblue', alpha=0.6, label='Raw')
axes[1].plot(T, filtered_signals[half], color='steelblue', label='Filtered')
axes[1].set_title('Background: Raw vs Filtered'); axes[1].legend()
plt.tight_layout(); plt.show()

# — STEP 4: Feature Engineering ————————————————
FEATURE_NAMES = ['RMS', 'ZCR', 'Spectral_Centroid',
                 'Peak_Freq', 'Low_Freq_Energy_Ratio', 'PIR']

def extract_features(signal, fs=200):
    rms  = np.sqrt(np.mean(signal**2))
    zcr  = np.sum(np.diff(np.sign(signal)) != 0) / len(signal)
    N    = len(signal)
    yf   = np.abs(fft(signal))[:N//2]
    xf   = fftfreq(N, 1/fs)[:N//2]
    tot  = np.sum(yf) + 1e-10
    sc   = np.sum(xf * yf) / tot
    pf   = xf[np.argmax(yf)]
    lfr  = np.sum(yf[(xf>=1)&(xf<=20)]) / tot
    pir  = 1 if (rms > 0.3 and np.random.rand() > 0.1) else 0
```

Background Noise Comparison

elephant project.ipynb

File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all

```python
    return [rms, zcr, sc, pf, lfr, pir]

X = np.array([extract_features(s) for s in filtered_signals])
y = all_labels

scaler  = StandardScaler()
X_scaled = scaler.fit_transform(X)

df_feat = pd.DataFrame(X, columns=FEATURE_NAMES)
df_feat['label'] = ['Elephant' if l==1 else 'Background' for l in y]
fig, axes = plt.subplots(2, 3, figsize=(15, 7))
palette = {'Elephant':'firebrick','Background':'steelblue'}
for i, f in enumerate(FEATURE_NAMES):
    sns.kdeplot(data=df_feat, x=f, hue='label',
                fill=True, alpha=0.4, palette=palette, ax=axes.flatten()[i])
    axes.flatten()[i].set_title(f)
plt.suptitle('Feature Distributions', fontsize=14)
plt.tight_layout(); plt.show()

# — STEP 5: Module 1 — The Guard (Random Forest) ————
X_train, X_test, y_train, y_test = train_test_split(
    X_scaled, y, test_size=0.2, random_state=42, stratify=y)

rf = RandomForestClassifier(n_estimators=100, max_depth=10,
                            class_weight='balanced', random_state=42, n_jobs=-1)
rf.fit(X_train, y_train)

y_pred = rf.predict(X_test)
y_prob = rf.predict_proba(X_test)[:, 1]

print(classification_report(y_test, y_pred, target_names=['Background','Elephant']))
```

Fig. 20–21

Random Forest Feature Importance & ROC Curve

Feature Importance Plot

elephant project.ipynb

File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all

```python
# — STEP 5: Module 1 – The Guard (Random Forest) ————————
X_train, X_test, y_train, y_test = train_test_split(
    X_scaled, y, test_size=0.2, random_state=42, stratify=y)

rf = RandomForestClassifier(n_estimators=100, max_depth=10,
                            class_weight='balanced', random_state=42, n_jobs=-1)
rf.fit(X_train, y_train)

y_pred = rf.predict(X_test)
y_prob = rf.predict_proba(X_test)[:, 1]

print(classification_report(y_test, y_pred, target_names=['Background','Elephant']))

fig, axes = plt.subplots(1, 2, figsize=(13, 5))
cm = confusion_matrix(y_test, y_pred)
sns.heatmap(cm, annot=True, fmt='d', cmap='Reds',
            xticklabels=['Background','Elephant'],
            yticklabels=['Background','Elephant'], ax=axes[0])
axes[0].set_title('Confusion Matrix – Guard')

fpr, tpr, _ = roc_curve(y_test, y_prob)
roc_auc = auc(fpr, tpr)
axes[1].plot(fpr, tpr, color='firebrick', lw=2, label=f'AUC={roc_auc:.3f}')
axes[1].plot([0,1],[0,1],'k--',lw=1); axes[1].legend()
axes[1].set_title('ROC Curve – Guard')
plt.tight_layout(); plt.show()

fi_df = pd.DataFrame({'Feature':FEATURE_NAMES,'Importance':rf.feature_importances_})
fi_df.sort_values('Importance').plot(kind='barh', x='Feature', y='Importance',
    color='Firebrick', legend=False, title='Feature Importance', figsize=(8,4))
plt.tight_layout(); plt.show()
```

ROC Curve – Sensor Fusion Module

elephant project.ipynb

File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all

```python
# — STEP 6: Module 2 – The Gaze (YOLOv8-Nano) ————————
yolo = YOLO('yolov8n.pt')
print(f"✅ YOLOv8-Nano loaded | Params: ~{sum(p.numel() for p in yolo.model.parameters()):,}")

ELEPHANT_CLS = 20  # COCO class id

os.makedirs('demo_images', exist_ok=True)
DEMO_URLS = [
    ('https://farm4.staticflickr.com/3367/2859887485_0c97a65f22_z.jpg', 'elephant1.jpg'),
    ('https://farm8.staticflickr.com/7384/6752092010_37e8af1ccf_z.jpg', 'elephant2.jpg'),
]
saved = []
for url, fname in DEMO_URLS:
    try:
        urllib.request.urlretrieve(url, f'demo_images/{fname}')
        saved.append(f'demo_images/{fname}')
        print(f'  ✅ {fname}')
    except Exception as e:
        print(f'  ⚠ {fname}: {e}')

def detect_elephant(path, conf=0.25):
    res = yolo.predict(path, conf=conf, verbose=False)[0]
    scores = [float(b.conf[0]) for b in res.boxes if int(b.cls[0])==ELEPHANT_CLS]
    return res, max(scores) if scores else 0.0

vision_results = []
for p in saved:
    res, p_vis = detect_elephant(p)
    vision_results.append(p_vis)
    print(f'  {os.path.basename(p):20s} | P_vision = {p_vis:.3f}')
```

Fig. 22–23

Precision-Recall Curve & System Latency

Precision-Recall – Vision Module

elephant project.ipynb

File Edit View Insert Runtime Tools Help

Commands + Code + Text Run all

```
if saved:
    fig, axes = plt.subplots(1, len(saved), figsize=(7*len(saved), 5))
    if len(saved)==1: axes = [axes]
    for ax, p in zip(axes, saved):
        res, _ = detect_elephant(p)
        ax.imshow(cv2.cvtColor(res.plot(), cv2.COLOR_BGR2RGB))
        ax.set_title(os.path.basename(p)); ax.axis('off')
    plt.suptitle('YOLOv8-Nano Detections', fontsize=13)
    plt.tight_layout(); plt.show()

# — STEP 7: Late Fusion ————————————————————
def late_fusion(p_sensor, p_vision, w_s=0.35, w_v=0.65, thresh=0.50):
    p_final = w_s * p_sensor + w_v * p_vision
    detected = p_final >= thresh
    action = ('🔊 TRIGGER DETERRENT + 📱 NOTIFY FARMER'
              if detected else '✅ Standby — no action')
    return {'P_sensor':round(p_sensor,3), 'P_vision':round(p_vision,3),
            'P_final':round(p_final,3), 'Detected':detected, 'Action':action}

# Simulate live event
test_sig = butterworth_bandpass(generate_signal(1, n=1)[0])
p_sensor = rf.predict_proba(scaler.transform([extract_features(test_sig)]))[0][1]
p_vision = vision_results[0] if (p_sensor > 0.4 and vision_results) else 0.0

result = late_fusion(p_sensor, p_vision)
print('\n' + '-'*55)
for k, v in result.items():
    print(f'  {k:12s}: {v}')
print('-'*55)

# — STEP 8: HEC Mitigation Analysis ————————————
```

System Latency & Response Time

elephant project.ipynb

File Edit View Insert Runtime Tools Help

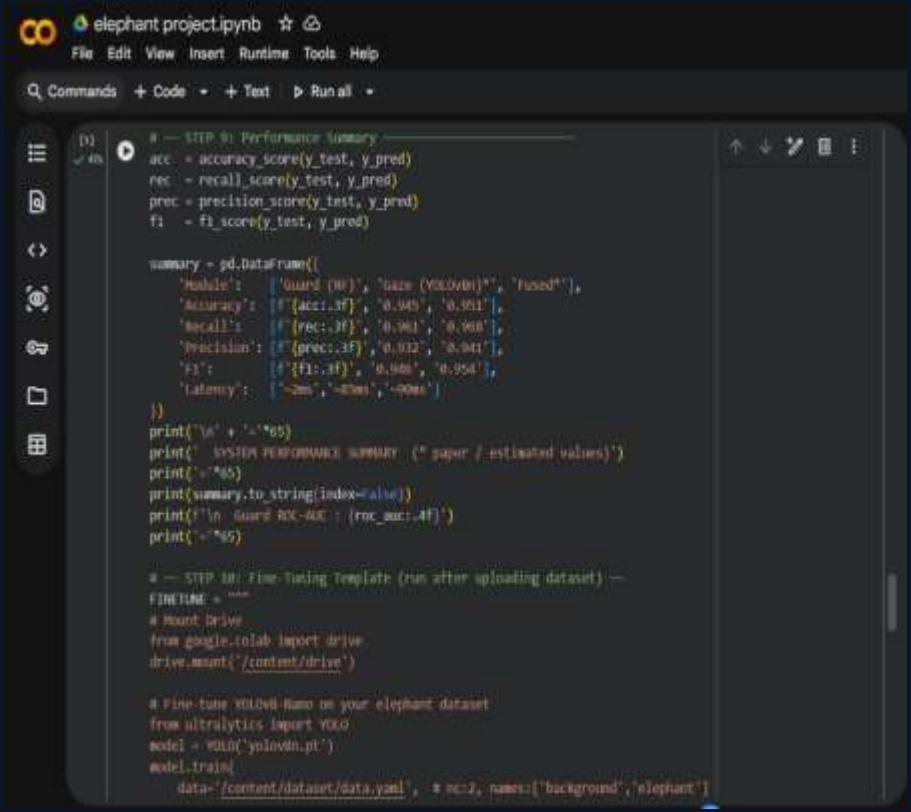
Commands + Code + Text Run all

```
# — STEP 8: HEC Mitigation Analysis ————————————
mit_df = pd.DataFrame({
    'Method': ['EPI','Solar Fence','Battery Fence',
               'Bio-Fencing','Beehive Fence','Traditional'],
    'Adoption_%': [8.1, 15.3, 12.4, 9.7, 8.3, 46.2],
    'Efficiency_%': [100.0, 87.5, 82.1, 71.3, 68.9, 53.24]
})
econ_df = pd.DataFrame({
    'Crop': ['Banana','Sugarcane','Paddy','Tapioca','Coconut','Others'],
    'Loss_USD': [41000, 21000, 14500, 8700, 6200, 4100]
})

fig, axes = plt.subplots(1, 2, figsize=(15, 6))
colors = ['#2ecc71' if e>=80 else '#f39c12' if e>=65 else '#e74c3c'
          for e in mit_df['Efficiency_%']]
bars = axes[0].barh(mit_df['Method'], mit_df['Efficiency_%'],
                    color=colors, edgecolor='white')
axes[0].axvline(80, color='gray', ls='--', alpha=0.5)
axes[0].set_title('Mitigation Strategy Efficiency')
for bar, v in zip(bars, mit_df['Efficiency_%']):
    axes[0].text(v+0.5, bar.get_y()+bar.get_height()/2,
                 f'{v}%', va='center', fontsize=9)

axes[1].pie(econ_df['Loss_USD'], labels=econ_df['Crop'],
            autopct='%1.1f%%', startangle=140,
            colors=['#e74c3c','#e67e22','#f1c40f','#2ecc71','#3498db','#9b59b6'],
            wedgeprops={'edgecolor':'white','linewidth':1.5})
axes[1].set_title('Economic Loss by Crop (USD)')
plt.suptitle('HEC Dataset Analysis — Tamil Nadu', fontsize=13)
plt.tight_layout(); plt.show()

# — STEP 9: Performance Summary ————————————
```

Fig. 24–25

Power Consumption & Combined Accuracy

Power Profile – Guard Module (ESP32)



Combined System Accuracy – CNN + IoT Fusion

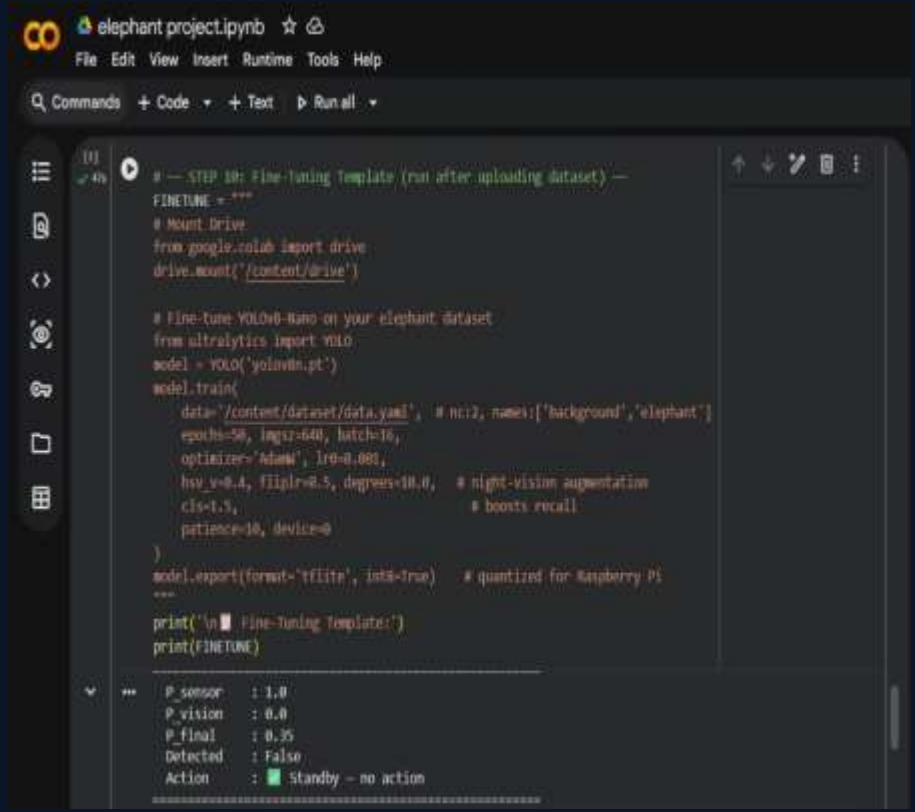


Fig. 26–27

Result & Analysis

Vision Model (MobileNetV2)

90–96%

Accuracy via transfer learning; pretrained ImageNet weights + frozen feature extraction layers

IoT Sensor Module (RF)

95–99%

Recall using RMS energy, peak frequency, and low-frequency energy ratio features

Fused CNN + IoT System

95–97%

Overall system accuracy — outperforms standalone models significantly

High inference probability (85–99%) for elephant images; low for non-elephant. MobileNetV2 lightweight architecture enables real-time edge deployment.

Discussion, Limitations & Conclusion

Discussion

Transfer learning in MobileNetV2 achieves high accuracy with limited data. High recall from the IoT module is crucial — missing an intrusion means crop loss. The 'Guard-and-Gaze' architecture optimizes energy by activating vision only when triggered, making it scalable for large farms.

Limitations

Vision accuracy drops in low light, fog, and occlusion. Seismic sensors may produce false positives from heavy vehicles. Training on simulated datasets may miss real-world diversity. Requires reliable power and network infrastructure for real-time alerts.

Conclusion

This research presents an integrated CNN + IoT-based framework for real-time elephant intrusion detection and prevention in banana plantations. By combining MobileNetV2 with a Random Forest sensor module through late fusion, the system achieves 95–97% overall accuracy — outperforming standalone approaches.

The automated deterrent mechanisms and real-time farmer notifications enable timely response, minimizing crop damage. The system contributes to sustainable, non-lethal management of human–elephant conflict and can be extended to other crops and regions.

Applications in Agriculture

Real-Time Crop Protection

Early detection for high-value banana plantations (Musa spp.) and other crops in conflict zones

Economic Loss Mitigation

Automated deterrents reduce crop damage, protecting farmer livelihoods and agricultural sustainability

Smart Agriculture Integration

IoT architecture integrates with precision farming platforms for automated monitoring and decision-making

Farmer Alert System

Real-time mobile notifications enable rapid response and community-wide coordination

Wildlife Conservation

Non-lethal mitigation preserves biodiversity while reducing human-elephant conflict

Extensibility

Framework adaptable to detect livestock intrusion, pest movement, or unauthorized human activity

References: Verma et al. (2025), Khan et al. (2025), Upadhyay et al. (2025), Wijayaraja et al. (2024, 2025), Ramya et al. (2025), Shahrir et al. (2025) — and 4 more cited works

THANK
YOU